Attendance Management System

Complete Technical Project & Viva Examination Handbook

An enterprise-grade desktop attendance logging, student directory, and academic eligibility monitoring system built with pure Java SE and Swing.

JAVA SE 17+ SWING GUI MVC ARCHITECTURE CSV STORAGE

1. Project Overview & Problem Statement

The **Attendance Management System** is a standalone desktop application developed in Java. It provides an automated, reliable alternative to traditional paper-based roll-call registers. The system manages student enrollment profiles, captures daily classroom attendance across departments, calculates institutional and individual attendance percentages in real-time, and automatically flags academic **defaulters** (students falling below the mandatory **75% attendance threshold**).

> **Key Academic Objectives**
>
> - Eliminate manual calculation errors and human tampering in attendance registers.
> - Provide immediate visual warnings and watchlist identification for students at risk of exam debarment.
> - Ensure zero external database installation dependency by using structured, persistent CSV storage.
> - Demonstrate clean Object-Oriented Programming (OOP) and Model-View-Controller (MVC) architectural separation.

2. Architectural Pattern: Model-View-Controller (MVC)

The project strictly implements the **Model-View-Controller (MVC)** design pattern, ensuring that data structures, user interfaces, and business algorithms are decoupled:

- **Model Layer (`model` , `exception`):** Encapsulates domain entities (`Student` , `Attendance`) and validation rules. It holds no UI logic or direct file read/write routines.
- **Controller / Business Layer (`manager`):** `AttendanceManager` acts as the single source of truth and mediator. It maintains the in-memory collections, handles data calculations (turnout rates, eligibility thresholds), and synchronizes state to disk.
- **View / Presentation Layer (`gui`):** Swing components (`Dashboard` , `StudentPanel` , `AttendancePanel` , `ReportPanel`) that capture user input and render visual data tables. The view never interacts directly with CSV files.

3. Packages Used & Responsibilities

3.1. Custom Application Packages

Package Name	Classes Contained	Architectural Responsibility
Default Package	`Main.java`	Application entry point, system Look-and-Feel configuration, high-DPI scaling, and EDT bootstrapping.
`model`	`Student.java` `Attendance.java`	Data models representing student entities and daily attendance records. Implements serialization and CSV mapping.
`manager`	`AttendanceManager.java` `StudentAttendanceSummary` (DTO)	Central business logic engine, thread synchronization, calculation algorithms, file I/O, and data queries.
`gui`	`Dashboard.java` `StudentPanel.java` `AttendancePanel.java` `ReportPanel.java` `StyledButton`	User interface layer built using Swing and AWT. Provides forms, tables, custom cell renderers, and navigation.
`exception`	`DuplicateStudentException.java`	Custom checked exception thrown during student enrollment if an ID collision is detected.

3.2. Standard Java SE Packages Utilized

Standard Library Package	Key Classes Used	Purpose in the Project
`javax.swing`	`JFrame`, `JPanel`, `JTabbedPane`, `JTable`, `JButton`, `JTextField`, `JComboBox`, `JSpinner`, `JProgressBar`, `JOptionPane`	Modern graphical user interface components, dialog boxes, and container controls.
`javax.swing.table`	`DefaultTableModel`, `DefaultTableCellRenderer`, `JTableHeader`, `TableColumn`	Customizing table models, cell editors, column dimensions, and conditional cell color rendering.
`java.awt`	`BorderLayout`, `GridBagLayout`, `GridLayout`, `FlowLayout`, `Graphics2D`, `Color`, `Font`	Component layouts, vector rendering of custom rounded buttons, antialiasing hints, and color definitions.
`java.time`	`LocalDate`, `DateTimeFormatter`	Immutable, thread-safe date parsing, formatting, and comparison without legacy `java.util.Date` pitfalls.
`java.io`	`BufferedReader`, `BufferedWriter`,	Reliable disk stream operations with explicit UTF-8 character encoding.

Standard Library Package	Key Classes Used	Purpose in the Project
	`FileInputStream`, `FileOutputStream`, `Serializable`	
`java.util`	`LinkedHashMap`, `ArrayList`, `TreeSet`, `Optional`, `Random`	In-memory data structures, fast key lookup, natural alphabetical sorting, and deterministic sample data seeding.
`java.util.stream`	`Stream`, `Collectors`, `Comparator`	Declarative filtering, counting, sorting, and transformation pipelines for reports and searches.

4. Detailed Class Breakdown

4.1. `Main.java` (Application Bootstrap)

- **Role:** Entry point containing `public static void main(String[] args)`.
- **Key Highlights:**
  - Enables system-level antialiasing and subpixel font rendering:

```
System.setProperty("awt.useSystemAAFontSettings", "on");
System.setProperty("swing.aatext", "true");
```

  - Applies the native OS Look-and-Feel via `UIManager.setLookAndFeel(...)`.
  - Instantiates `AttendanceManager` to load data into memory.
  - Launches `Dashboard` on Swing's **Event Dispatch Thread (EDT)** via `SwingUtilities.invokeLater()`.

4.2. `model.Student`

- **Role:** Encapsulates individual student profile data.
- **Attributes:** `studentId`, `name`, `email`, `department`, `semester`.
- **Key Methods:**
  - `toCsv()`: Formats fields as a comma-separated string, escaping special characters.
  - `fromCsv(String line)`: Robustly parses CSV strings using a regex lookahead that preserves quoted commas.
  - `equals(Object o)` & `hashCode()`: Overridden based on case-insensitive `studentId` to guarantee uniqueness in collections.
  - Implements `Serializable` for object state transfer.

4.3. `model.Attendance`

- **Role:** Represents an attendance record for a student on a specific date.
- **Constants:** `STATUS_PRESENT` ("Present"), `STATUS_ABSENT` ("Absent"), `STATUS_LATE` ("Late"), `STATUS_EXCUSED` ("Excused").
- **Attributes:** `studentId`, `date` (`LocalDate`), `status`, `remarks`.
- **Key Methods:**
  - `normalizeStatus(String status)`: Sanitizes status inputs against official constants.
  - `toCsv()` / `fromCsv()`: Serializes to `StudentID,YYYY-MM-DD,Status,Remarks`.
  - `equals()` & `hashCode()`: Composite uniqueness based on `studentId + date` (one record per student per day).

4.4. `exception.DuplicateStudentException`

- **Role:** Custom checked exception inheriting from `java.lang.Exception`.
- **Purpose:** Thrown when an administrator attempts to register a student ID that already exists in the system. Enforces compile-time handling so that UI layers display an informative dialog instead of failing silently.

4.5. `manager.AttendanceManager` (Core Business Engine)

- **In-Memory Collections:**

```
private final Map<String, Student> studentMap = new LinkedHashMap<>();
private final List<Attendance> attendanceList = new ArrayList<>();
```

  Why LinkedHashMap? Provides O(1) constant-time search/updates by student ID while preserving insertion order for UI tables.
- **Thread Safety:** All public mutator and query methods are marked `synchronized` to ensure concurrent access safety.
- **Key Operations:**
  - `addStudent(Student s)` : Validates inputs, checks for duplicate IDs, and updates `students.csv` .
  - `deleteStudent(String id)` : Removes the student and executes **cascade deletion** on their associated attendance records.
  - `searchStudents(String query)` : Multi-field live search matching ID, name, email, department, or semester using Java Streams.
  - `recordAttendance(...)` : Upsert logic (updates status if record exists for student+date, otherwise appends new record).
  - `getAttendanceSummaries(...)` : Aggregates metrics (total, present, absent, late, excused, %) across specified date ranges and department filters.
  - `exportReportToCsv(...)` : Exports academic audits to formatted spreadsheet-compatible CSV files.
  - `seedSampleData()` : Automatically generates 12 students and 10 days of attendance history on first launch.

4.6. `AttendanceManager.StudentAttendanceSummary` (DTO)

- **Role:** Static inner Data Transfer Object holding computed metrics for a student.
- **Attributes:** `student` , `totalClasses` , `presentCount` , `absentCount` , `lateCount` , `excusedCount` , `percentage` .
- **Eligibility Rules:**
  - **Eligible:** Attendance ≥ 75%
  - **Warning:** 65% ≤ Attendance < 75%
  - **Defaulter / Barred:** Attendance < 65%

4.7. GUI Components Breakdown

- **`Dashboard.java` :** Main `JFrame` with header branding, system date, status footer, `JTabbedPane` , 4 KPI cards, live turnout progress bar, and Defaulters Watchlist table. Contains `notifyDataChanged()` callback.
- **`StyledButton` :** Custom component extending `JButton` that overrides `paintComponent()` with `Graphics2D` to deliver high-contrast buttons with rounded corners and hover effects.
- **`StudentPanel.java` :** Left-right `JSplitPane` holding the student registration form (`GridBagLayout`) and the searchable student directory table with custom `AttendancePercentageRenderer` .
- **`AttendancePanel.java` :** Daily roll-call sheet featuring a date spinner, quick "Today"/"Yesterday" buttons, department filters, bulk mark actions, in-table `JComboBox` status editor, and color-coded `StatusCellRenderer` .
- **`ReportPanel.java` :** Auditing interface with date-range filters, defaulter-only toggle, 4 KPI summary cards, full academic eligibility table, and CSV export via `JFileChooser` .

5. Data Storage & Persistence Engine

The system stores data in comma-separated values (CSV) format inside the `./data/` directory:

- `data/students.csv`: `StudentID,Name,Email,Department,Semester`
- `data/attendance.csv`: `StudentID,Date,Status,Remarks`

> **Regex CSV Parsing Technique**
>
> To safely handle values that contain commas (e.g. names or remarks like "*Doe, John*"), the project avoids basic `split(",")` and instead uses a regular expression lookahead:
>
> ```
> line.split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)", -1);
> ```
>
> This ensures splits only occur on commas that are outside matching pairs of double quotation marks.

6. Core Computer Science & Java Concepts Applied

1. **Object-Oriented Programming (OOP):**
   - *Encapsulation:* Private attributes accessed strictly via public getters/setters with validation.
   - *Inheritance:* Custom panels extend `JPanel`; custom exceptions extend `Exception`.
   - *Polymorphism:* Overriding `getTableCellRendererComponent()` and `paintComponent()`.
   - *Abstraction:* GUI components interact with `AttendanceManager` without needing to know file storage details.
2. **Java Collections Framework:**
   - `LinkedHashMap` for O(1) indexed lookups while preserving insertion order.
   - `ArrayList` for ordered attendance logs.
   - `TreeSet` with `String.CASE_INSENSITIVE_ORDER` for sorted unique department lists.
3. **Modern Java 8+ Streams & Functional Lambdas:**
   - Stream filtering, counting, mapping, and collectors for analytics, defaulters detection, and live searching.
4. **Concurrency & Event Dispatch Thread (EDT):**
   - Thread-safe data access using `synchronized` keyword.
   - Safe Swing component creation via `SwingUtilities.invokeLater()`.
5. **Defensive File I/O & Exception Handling:**
   - Try-with-resources with explicit UTF-8 character encoding to prevent resource leaks and character corruption.
   - Checked custom exception `DuplicateStudentException`.

7. Comprehensive Viva Questions & Model Answers

Q1: What is the main objective of this project?

The Attendance Management System is a desktop application developed in Java SE to automate student attendance tracking, enrollment management, and academic eligibility auditing. It provides real-time defaulter detection (students below 75% attendance), batch marking, and one-click CSV report exports.

Q2: Which architectural pattern does the application follow?

It follows the **Model-View-Controller (MVC)** architectural pattern: **Model:** `Student` and `Attendance`. **View:** Swing GUI classes (`Dashboard`, `StudentPanel`, `AttendancePanel`, `ReportPanel`). **Controller:** `AttendanceManager`, which controls business logic, calculations, and data persistence.

Q3: Why did you use SwingUtilities.invokeLater() in Main.java?

Swing UI components are not thread-safe. Modifying or instantiating them from the initial main thread can cause race conditions or UI freezes. `SwingUtilities.invokeLater()` places GUI tasks onto Swing's dedicated **Event Dispatch Thread (EDT)**.

Q4: Why did you use LinkedHashMap instead of a standard HashMap or ArrayList for students?

`LinkedHashMap` provides $O(1)$ constant-time lookup, insertion, and deletion by `studentId`. In addition, it maintains a doubly-linked list running through all entries, which **preserves insertion order** so students appear predictably in the UI tables.

Q5: How do you handle duplicate student IDs?

We implemented a custom checked exception named `DuplicateStudentException`. When `addStudent()` is called in `AttendanceManager`, it checks if the upper-cased student ID already exists in `studentMap`. If so, it throws the exception, which `StudentPanel` catches to display an informative error dialog via `JOptionPane`.

Q6: Why is DuplicateStudentException a checked exception rather than an unchecked exception?

Checked exceptions extend `java.lang.Exception` and enforce compile-time verification. Enrolling a duplicate student ID is a foreseeable user error that the presentation layer must anticipate, catch, and handle gracefully without crashing the application.

Q7: How does your CSV parser handle values containing commas (e.g. "Doe, John")?

Standard `line.split(",")` breaks if fields contain commas. Our `fromCsv()` method uses a regular expression lookahead: `line.split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)", -1)`. This splits only on commas followed by an even number of quotation marks, indicating the comma is outside quotes.

Q8: What happens to attendance history when a student is deleted?

`AttendanceManager.deleteStudent()` implements **cascade deletion**: it removes the student from `studentMap` and immediately invokes `attendanceList.removeIf(a -> a.getStudentId().equalsIgnoreCase(studentId))`, saving both updated files to disk to prevent orphaned records.

Q9: How is the attendance percentage calculated? Are late arrivals counted?

Attendance percentage is calculated as: `((Present Count + Late Count) / Total Classes) * 100`. Late arrivals are counted as attended (with a late flag), while Absent and Excused contribute to total conducted classes without incrementing the attended count.

Q10: What are the academic eligibility status rules in your system?

- **Eligible (≥ 75%):** Green badge; permitted for exams.
- **Warning (65% to 74.9%):** Amber badge; attendance warning issued.
- **Defaulter / Barred (< 65%):** Red badge; strictly debarred from examinations.

Q11: Why are methods in AttendanceManager marked synchronized?

The `synchronized` keyword ensures thread safety. It enforces mutual exclusion so that if multiple UI events, background tasks, or timers access shared collections (`studentMap` , `attendanceList`) concurrently, race conditions and data corruption are prevented.

Q12: Why use java.time.LocalDate instead of the traditional java.util.Date?

`java.util.Date` is mutable, not thread-safe, and uses 0-indexed months. `LocalDate` (introduced in Java 8) is immutable, thread-safe, and strictly represents a calendar date (YYYY-MM-DD) without time or timezone complexities.

Q13: How does the in-table dropdown editor work in AttendancePanel?

In `AttendancePanel` , the 4th column of `JTable` is assigned a `DefaultCellEditor` wrapping a `JComboBox` : `colStatus.setCellEditor(new DefaultCellEditor(statusEditorCombo))` . This enables teachers to change a student's status via an interactive dropdown directly inside table cells.

Q14: How do table cells dynamically render colored text (e.g. red for defaulters)?

By implementing custom renderers extending `DefaultTableCellRenderer` and overriding `getTableCellRendererComponent()` . The method inspects the cell value, parses the percentage or status, and calls `setForeground(COLOR_DANGER)` or `setForeground(COLOR_SUCCESS)` accordingly.

Q15: What design pattern is used to update the Dashboard when data changes in other tabs?

The **Observer / Callback pattern**. Child panels retain a reference to `Dashboard` . After saving attendance or adding students, they invoke `parentDashboard.notifyDataChanged()` , which recalculates the KPI cards, turnout rate, and Defaulters Watchlist.

Q16: Which layout managers did you use and why?

`BorderLayout` for overarching panel divisions; `GridBagLayout` for the student registration form to guarantee precise grid alignment across screen resolutions; `GridLayout` for evenly spaced KPI cards; and `FlowLayout` for horizontal filter toolbars.

Q17: Why did you build StyledButton instead of using standard JButton?

Standard `JButton` components are subject to operating system Look-and-Feel overrides, often yielding flat or low-contrast gray buttons. `StyledButton` uses custom 2D vector painting (`Graphics2D`) with antialiasing to guarantee high-contrast colors, rounded corners (6px), and hover/press visual feedback.

Q18: What is the purpose of serialVersionUID in the model classes?

`Student` and `Attendance` implement `Serializable` . `serialVersionUID` is a unique version identifier used during deserialization to verify that the loaded class definition matches the serialized object stream, avoiding `InvalidClassException` .

Q19: How did you implement multi-field search in StudentPanel?

Using Java 8 Streams: `studentMap.values().stream().filter(...)` tests case-insensitive partial substring matches against student ID, name, department, email, and semester simultaneously.

Q20: What are the advantages of CSV storage in this project over a full SQL database?

1. Zero configuration & portable: runs out-of-the-box on any machine with a JDK without needing MySQL installation or JDBC drivers.
2. Demonstrates core Java file stream I/O and regex parsing capabilities.
3. Easy data inspection and Microsoft Excel compatibility.

Q21: How does sample data seeding work?

On startup, `AttendanceManager.loadData()` checks if `studentMap` is empty. If no files exist, `seedSampleData()` provisions 12 realistic student records across 4 engineering departments and 10 days of deterministic attendance history using a seeded `Random(42)` instance.

Q22: If you had another sprint to enhance this system, what would you add?

1. Database integration via JDBC/Hibernate for multi-user client-server deployments.
2. Role-based authentication (Admin vs. Faculty vs. Student view).
3. Automated email notifications using JavaMail API for low-attendance alerts.
4. Graphical attendance charts using JFreeChart.

8. Rapid Revision Cheat Sheet

Topic / Area	Key Points for Instant Recall
Platform & Tools	Java SE 17+, Swing & AWT GUI, Java Time API, Java Streams API.
Architecture	**Model-View-Controller (MVC)** separation.
Data Persistence	CSV files in `./data/` (`students.csv` , `attendance.csv`) using UTF-8 `BufferedReader` / `BufferedWriter` .
Collections Used	`LinkedHashMap` (O(1) search + maintains insertion order), `ArrayList` (chronological attendance), `TreeSet` (alphabetical unique departments).
Exception Handling	`DuplicateStudentException` (Custom checked exception for ID collisions).
Attendance Formula	`((Present + Late) / Total Classes) * 100%` .
Eligibility Bar	**75.0%** mandatory minimum threshold for exam eligibility.
Thread Safety	Methods in `AttendanceManager` are `synchronized` ; GUI starts on Swing's EDT via `SwingUtilities.invokeLater()` .
GUI Modules	`Dashboard` (Overview + Watchlist), `StudentPanel` (CRUD), `AttendancePanel` (Daily Roll Call), `ReportPanel` (Auditing & Export).
CSV Regex Lookahead	`split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)", -1)` splits on commas outside quotes.